42 MULHOUSE

Makefile

eschwart

Petit projet en C

Trois fichiers : main.c, parser.c, utils.c. Pour compiler :

```
cc -Wall -Wextra -Werror main.c parser.c utils.c -o program
```

Commande longue **Facile de se tromper** **Tout est recompilé à chaque fois**

Votre premier Makefile

Makefile

```
program:
    cc main.c utils.c parser.c -o program
```

Terminal

```
$ make
```

make exécute la première règle

La commande commence par une TAB

Pas des espaces, sinon : missing separator.

Les dépendances

Si un fichier est plus récent que sa cible, make relance la règle.

```
program: main.o utils.o parser.o
    cc main.o utils.o parser.o -o program

main.o: main.c
    cc -c main.c -o main.o

utils.o: utils.c
    cc -c utils.c -o utils.o

parser.o: parser.c
    cc -c parser.c -o parser.o
```

```
program
├── main.o
│   └── main.c
├── utils.o
│   └── utils.c
└── parser.o
    └── parser.c
```

Seul ce qui a changé est recompilé.

Attention au relink

La cible doit porter le nom du fichier produit.

Makefile

```
program:
    cc main.c -o program_name
```

Terminal

```
$ make
cc main.c -o program_name
$ make
cc main.c -o program_name
```

> Ici, aucun fichier « program » n'existe : tout est refait à chaque make. Un 2ᵉ make doit répondre **Nothing to be done**.

Les règles génériques

Une seule règle pour tous les fichiers .o

Avant

```
main.o: main.c
    cc -c main.c -o main.o
utils.o: utils.c
    cc -c utils.c -o utils.o
parser.o: parser.c
    cc -c parser.c -o parser.o
```

Après

```
%.o: %.c
    cc -c $< -o $@
```

`%` la partie variable du nom

`$@` la cible (main.o)

`$<` la 1re dépendance (main.c)

Le linking avec make

Le programme dépend de tous les .o : si un seul change, make refait le lien.

Makefile

```
program: main.o utils.o parser.o
    cc $^ -o $@
```

`$@` la cible

`$^` toutes les dépendances

Ce que make lance

```
cc main.o utils.o parser.o -o program
```

Les variables

On écrit une valeur une fois, on l'utilise partout.

Makefile

```
CC      = cc
CFLAGS = -Wall -Wextra -Werror

%.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@

program: main.o utils.o parser.o
    $(CC) $^ -o $@
```

Opérateur	Effet
=	valeur calculée à chaque utilisation
:=	valeur calculée une fois
?=	seulement si pas déjà définie
+=	ajoute à la suite

Générer la liste des .o

$(SRC:.c=.o) remplace chaque .c par .o

Makefile

```
SRC = main.c utils.c parser.c
OBJ = $(SRC:.c=.o)

program: $(OBJ)
    $(CC) $(OBJ) -o $@

%.o: %.c
    $(CC) $(CFLAGS) -c $< -o $@
```

Résultat

```
OBJ = main.o utils.o parser.o
```

Ajouter un fichier

Une seule ligne à modifier : SRC.

Les directives spéciales

```
.PHONY: clean
```

.PHONY

La cible n'est pas un fichier : la règle est toujours exécutée.

```
clean:
	@rm -f $(OBJ)
```

@

Cache une commande, mais garde son résultat.

```
.SILENT:
```

.SILENT

Cache toutes les commandes. À éviter pour débuter.

Un Makefile complet

Les règles demandées à 42 : $(NAME), all, clean, fclean, re

```
NAME   = program
CC     = cc
CFLAGS = -Wall -Wextra -Werror
SRC    = main.c utils.c parser.c
OBJ    = $(SRC:.c=.o)

all: $(NAME)

$(NAME): $(OBJ)
    $(CC) $(CFLAGS) $(OBJ) -o $(NAME)
```

```
clean:
    rm -f $(OBJ)

fclean: clean
    rm -f $(NAME)

re: fclean
    $(MAKE) all

.PHONY: all clean fclean re
```

Segmenter un Makefile

include colle le contenu d'un autre fichier, comme #include en C.

Makefile

```
include mk/variables.mk
include mk/sources.mk
include mk/rules.mk
```

mk/sources.mk

```
SRC = src/main.c \
      src/parser.c \
      src/utils.c
```

```
project/
├── Makefile
├── mk/
│   ├── variables.mk
│   ├── sources.mk
│   └── rules.mk
├── src/
└── include/
```

Automatiser des tâches

make ne sert pas qu'à compiler : c'est aussi un task runner.

Makefile

```
test:
    python3 tester/tester.py

valgrind:
    valgrind --leak-check=full ./$(NAME)

norm:
    norminette

run:
    ./$(NAME)
```

Exemples :

- lancer un testeur
- lancer Valgrind
- vérifier la norme
- git add / commit / push

Pense à les mettre dans .PHONY.

Automatiser les tests

Makefile

```
test:
    ./tests/run_tests.sh
```

Terminal

```
$ make test
```

Une seule commande à retenir

Le même comportement pour tout le monde

Du temps gagné en développement

Automatiser Git

Makefile

```
push:
	git add .
	git commit -m "$(MSG)"
	git push
```

Terminal

```
$ make push MSG="fix parsing"
```

MSG est une variable

On la donne au lancement de make.

Ajoute un .gitignore

Sinon les .o et le binaire partent aussi.

Automatiser Git ++

Une cible peut regrouper tout un workflow : branche, commit, push.

Makefile

```
push:
    @git status --short
    @printf 'Go ? [y/N] '; read ans; \
    if [ "$$ans" = "y" ]; then \
        git add . && git commit && git push; \
    fi
```

$$ = un $ pour le shell

\ = même ligne de shell

Sinon la variable est perdue.

> Une cible qui fait **git reset --hard** efface du travail : toujours demander confirmation.

Petits bonus

Ranger les sources par dossier

addprefix ajoute le chemin devant chaque fichier.

Makefile

```
CONFIG_FILES = Config.cpp Location.cpp ServerConfig.cpp
HTTP_FILES   = HttpRequest.cpp HttpResponse.cpp

CONFIG = $(addprefix srcs/config/, $(CONFIG_FILES))
HTTP   = $(addprefix srcs/http/, $(HTTP_FILES))
```

Résultat

```
HTTP = srcs/http/HttpRequest.cpp srcs/http/HttpResponse.cpp
```

Compiler en parallèle

Un job par cœur du processeur : beaucoup plus rapide.

```
MAKEFLAGS += -j$(shell nproc)
```

Les dépendances doivent être exactes

Sinon deux règles liées peuvent tourner en même temps.

re doit appeler $(MAKE) all

Et pas dépendre de $(NAME), sinon fclean et la compilation se chevauchent.

Encore trois astuces

```
.DEFAULT_GOAL := all
```

La règle par défaut

Celle que lance make tout court.

```
build:
	@printf "Compiling...\n"
```

Des messages

Pour suivre les étapes du build.

```
CFLAGS += -MMD -MP
-include $(OBJ:.o=.d)
```

Suivre les .h

Un header modifié recompile les .c qui l'utilisent.

Merci !

eschwart · 42 Mulhouse